COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	October 6, 2026	**DATE SUBMITTED:**	October 8, 2026
NAME/S:	Benj Federic C. De Leon C. Mendoza, Ryo Shin P.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11.1 Basic Algorithm Analysis

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
    if(search(x, y[i])!= - 1){
        return false;
    }
    }
    return true;
}
```

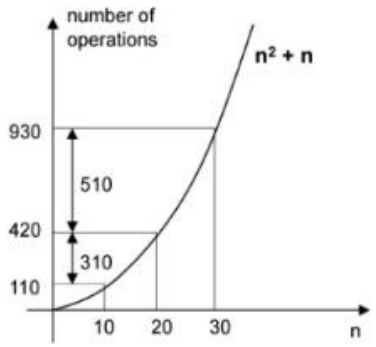
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: **mxn**

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worst case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis	
DE LEON, BENJ FEDERIC	As the input (n) grows, the required work speeds up: 10 items take 110 steps, 20 take 420, and 30 take 930. The jumps between these numbers get bigger each time (310, then 510), showing the growth isn't a simple straight line.
MENDOZA, RYO SHIN	The operations increase rapidly as nn grows: 110 for n=10n=10, 420 for n=20n=20, and 930 for n=30n=30. This shows quadratic growth because of the n2n^2 term. Therefore, the time complexity of diff is $O(n^2)$, which is more suitable for smaller inputs.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

main.cpp



```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

//STEP 1
class Student{
    private:
        std::pair<int, int> name;
        bool vaccinated;

    public:
        //constructor
        Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){
        }

        // Getters
        auto get_name() { return name; }
        auto is_vaccinated() { return vaccinated; }


//STEP 2
        // Two people are same if they have the same name
        bool operator ==(const Student& p) const {
            return this->name == p.name;
        }

        // The ordering of a set of people is defined by their name
        bool operator< (const Student& p) const {
            return this->name < p.name;
        }

        bool operator> (const Student& p) const {
            return this->name > p.name;
        }
};


//STEP 3
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());

    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}
```

```cpp
//function that implements the logic for whether a vaccination is needed
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true)
    {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
        else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);
        if (mid_element < P)
            std::advance(first, mid_element_index);

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}




//use of std::chrono for experimental analysis.
void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "Time taken to search = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;
    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
<< ") " << "needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
<< ") " << "does not need vaccination." << std::endl;
}
```



```
1  //driver code
2  int main() {
3      // Generate a Student to search
4      auto p = generate_random_Student(1000);
5      search_test(1000, p);
6      search_test(10000, p);
7      search_test(100000, p);
8      return 0;
9  }
```

Output

```
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA11p1> c
($?) { g++ main.cpp -o main } ; if ($?) { .\main }
Time taken to search = 1 microseconds
Student (939 53) needs vaccination.
Time taken to search = 1 microseconds
Student (939 53) needs vaccination.
Time taken to search = 4 microseconds
Student (939 53) needs vaccination.
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA11p1>
```

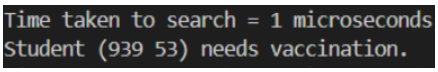
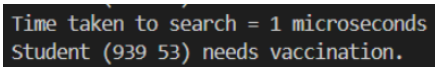
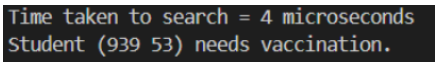
Input SIze (N)	Execution Speed	Screenshot	Observation(s)
1.000	1 microsend		Same student (939 53) needs vaccination
10,000	1 microsend		Same student; time almost unchanged
100,000	4 microsends		Same student; time only slightly higher

Table 11.1

Analysis	
DE LEON, BENJ FEDERIC	Increasing the input 100 times barely changes search time, which stays at just a few microseconds. This shows how fast binary search is, it cuts the search area in half each time, taking about 10 steps for 1,000 items and only 17 steps for 100,000 items.
MENDOZA, RYO SHIN	Binary search remains fast even when the input size increases from 1,000 to 100,000 because it repeatedly halves the search range. This shows its **O(log n)** efficiency. Missing names take the most steps, but the search is still much faster than linear search.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis.

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Theoretical Analysis:

Number of comparisons:: n(n-1)/2
Time Complexity: O(n^2)
Base Case: O(1)
Worst Case: O(n^2)

Array Size n	Maximum Comparisons
10	45
100	4,950
1,000	499,500
10,000	49,995,000

```cpp
//Problem 1: Unique(A)
long long comparisons = 0;

bool unique_elements(std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n - 1 + 1; i++) {
        for (int j = i + 1; j < n; j++) {
            comparisons++;
            if (A[i] == A[j])
                return false;
        }
    }
    return true;
}
```

Analysis
The algorithm is simple and requires no extra storage, but its O(n^2) time complexity makes it inefficient for large arrays. In the worst case, every pair of elements must be compared before returning true. If a duplicate is found early, however, the algorithm can terminate much sooner.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

```cpp
//Problem 2: power functions
long long calls = 0;

unsigned long long rpower(unsigned long long x, int n) {
    calls++;
    if (n == 0) return 1;
    else return x * rpower(x, n - 1);
}

unsigned long long brpower(unsigned long long x, int n) {
    calls++;
    if (n == 0) return 1;
    unsigned long long y;
    if (n % 2 == 1) {
        y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        y = brpower(x, n / 2);
        return y * y;
    }
}
```

supplementary.cpp



```cpp
#include <iostream>
#include <vector>
#include <chrono>

//Problem 1: Unique(A)
long long comparisons = 0;

bool unique_elements(std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n - 1 + 1; i++) {
        for (int j = i + 1; j < n; j++) {
            comparisons++;
            if (A[i] == A[j])
                return false;
        }
    }
    return true;
}

//Problem 2: power functions
long long calls = 0;

unsigned long long rpower(unsigned long long x, int n) {
    calls++;
    if (n == 0) return 1;
    else return x * rpower(x, n - 1);
}

unsigned long long brpower(unsigned long long x, int n) {
    calls++;
    if (n == 0) return 1;
    unsigned long long y;
    if (n % 2 == 1) {
        y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    using clk = std::chrono::steady_clock;
    using us = std::chrono::microseconds;

    std::cout << "PROBLEM 1: Unique(A) - worst case (all elements unique)\n";
    std::cout << "n\tcomparisons\tn(n-1)/2\ttime(us)\n";
    for (int n : {1000, 2000, 4000, 8000, 16000}) {
        std::vector<int> A(n);
        for (int i = 0; i < n; i++) A[i] = i;
        comparisons = 0;
        auto b = clk::now();
        bool r = unique_elements(A);
        auto e = clk::now();
        std::cout << n << "\t" << comparisons << "\t\t" << (long long)n * (n - 1) / 2
                  << "\t\t" << std::chrono::duration_cast<us>(e - b).count()
                  << (r ? "" : " (not unique)") << "\n";
    }

    std::cout << "\nPROBLEM 2: rpower vs brpower (x = 1 to avoid overflow)\n";
    std::cout << "n\trpower calls\trpower(us)\tbrpower calls\tbrpower(us)\n";
    for (int n : {1000, 2000, 4000, 8000, 16000}) {
        calls = 0;
        auto b1 = clk::now();
        rpower(1, n);
        auto e1 = clk::now();
        long long c1 = calls;

        calls = 0;
        auto b2 = clk::now();
        brpower(1, n);
        auto e2 = clk::now();
        long long c2 = calls;

        std::cout << n << "\t" << c1 << "\t\t"
                  << std::chrono::duration_cast<us>(e1 - b1).count() << "\t\t"
                  << c2 << "\t\t"
                  << std::chrono::duration_cast<us>(e2 - b2).count() << "\n";
    }
    return 0;
}
```

Output

```
1\HoA11p1\" ; if ($?) { g++ activity11_supplementary.cpp -o activity11_supplementar
PROBLEM 1: Unique(A) - worst case (all elements unique)
n       comparisons     n(n-1)/2        time(us)
1000    499500          499500          7010
2000    1999000         1999000         25302
4000    7998000         7998000         105333
8000    31996000                31996000                441887
16000   127992000               127992000               1656953

PROBLEM 2: rpower vs brpower (x = 1 to avoid overflow)
n       rpower calls    rpower(us)      brpower calls   brpower(us)
1000    1001            86              11              0
2000    2001            66              12              0
4000    4001            116             13              0
8000    8001            224             14              0
16000   16001           395             15              0
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA11p1>
```

Conclusion	
DE LEON	Through this activity, we learned how theoretical algorithm analysis directly corresponds to experimental runtime performance, confirming that Big-O time complexity dictates how algorithms scale as input sizes grow.
MENDOZA, RYO	In this activity, we learned how to analyze algorithm efficiency using theoretical and experimental analysis. We learned that an algorithm's running time depends on the input size and its growth rate. The supplementary activity also showed that quadratic and linear algorithms take more time as the input increases, while logarithmic algorithms are more efficient.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: *Code assistance, explanation of concepts, and summarization, grammar checking*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Gemini was used to find/fix some bugs and guide us through the supplementary code, but we reviewed and understood it.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.